

INT305 Coursework 2: Image Object Classification (CIFAR-10)

Name: Nan Lin

Student ID: 2252173

Problem 1

1. Please describe the 2 key components in the CNN framework: the convolutional kernel (**10'**) and the loss functions used in the framework (**10'**).

Answer:

Convolutional Kernel

A convolutional kernel is a small learnable matrix that slides across an input image or feature map to extract local spatial patterns. For each spatial location, the kernel performs element-wise multiplication with the corresponding receptive field and sums the results to produce an output activation. Formally, for an input feature map $X \in \mathbb{R}^{H \times W}$ and a kernel $K \in \mathbb{R}^{k \times k}$, the convolution is defined as:

$$Y(i, j) = \sum_{m=1}^k \sum_{n=1}^k K(m, n) X(i + m - 1, j + n - 1),$$

where $Y(i, j)$ denotes the output feature at position (i, j) .

In practical CNNs, convolution is often combined with *stride* (the step size of the kernel) and *padding* (zero-border additions) to control the spatial resolution of the output feature maps. Because kernel weights are shared across spatial locations, the convolution operation enjoys parameter efficiency and translation invariance. By stacking multiple convolutional layers, CNNs can build hierarchical representations, progressing from low-level edges and textures to high-level semantic concepts.

Loss Functions in CNN Classification

For multi-class image classification, CNNs are commonly trained using the *cross-entropy loss*, which measures the discrepancy between the predicted probability distribution and the true class label. Given a logit vector $\mathbf{z}$ and its softmax probabilities

$$p_c = \frac{e^{z_c}}{\sum_{j=1}^C e^{z_j}},$$

the cross-entropy loss for a sample with true class y is

$$\mathcal{L}_{\text{CE}} = -\log p_y.$$

For a batch of N samples, the average loss is

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \log p_{y_i}.$$

This loss encourages the network to assign high probability to the correct class and produces larger gradients when the model is confidently wrong, which accelerates learning and stabilises training. Compared with alternatives such as mean squared error (MSE) or hinge loss, cross-entropy aligns more naturally with probability-based outputs and is therefore the standard choice for CNN-based image classification tasks.

Problem 2

2. Please train (or fine-tune) and test the framework (provided below references) on CIFAR-10 and report the final accuracy performance that you have achieved. You need to explain the environment of your code and training steps in the report (5'). You also need to show the inference steps in the report (5'). Please also report some well classified and misclassified images by including the images and corresponding classification confidence value and the evaluation metrics used in your discussion (10') and comment on your findings (20').

Answer:

Code Running Environment

All experiments were executed on a macOS device equipped with Apple Silicon GPU, using the Metal Performance Shaders (MPS) backend provided by PyTorch. The device is selected automatically via:

```
# 1. Choose Device (MPS for Mac, otherwise CPU)
if torch.backends.mps.is_available():
    device = torch.device("mps")
    print("Using device: MPS")
else:
    device = torch.device("cpu")
    print("Using device: CPU")
```

Figure 1: Code running environment.

If MPS is available, both the model and all tensors are moved to the MPS device using `to(device)`; otherwise, the program falls back to CPU execution. The implementation uses Python, PyTorch, and Torchvision, and the CIFAR-10 dataset is downloaded automatically into the local `./data` directory.

Data preprocessing follows standard CIFAR-10 normalization, where each RGB channel is mapped to:

$$x_{\text{norm}} = \frac{x - 0.5}{0.5}.$$

Training and testing sets are loaded with a batch size of 4. Due to multiprocessing limitations on macOS, the dataloaders use `num_workers = 0`. Shuffling is enabled only for the training loader.

Training Steps

In this subsection I summarise the main training steps used to optimise the CNN on the CIFAR-10 training set. For each step, I also attach the corresponding code snippet as a screenshot and briefly explain why it is important for solving the problem.

1. Step 1 – Initialise the CNN and move it to the selected device.

The network architecture is defined in the `Net` class, and the instance is moved to the device determined in the environment setup using `net = Net().to(device)`. This step ensures that all subsequent computations (forward and backward passes) are executed either on the Apple MPS GPU or on the CPU.

```
# 3. CNN Model (Baseline Teacher Version)
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.conv1 = nn.Conv2d(3, 6, 5)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(6, 16, 5)
        self.fc1 = nn.Linear(16*5*5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)
```

Figure 2: Figure 4: Definition of the cross-entropy loss and SGD optimiser.

```
def forward(self, x):
    x = self.pool(F.relu(self.conv1(x)))
    x = self.pool(F.relu(self.conv2(x)))
    x = x.view(-1, 16*5*5)
    x = F.relu(self.fc1(x))
    x = F.relu(self.fc2(x))
    x = self.fc3(x)
    return x
```

Figure 3: Device selection logic for switching between MPS and CPU execution.

2. Step 2 – Define loss function and optimiser.

I use `nn.CrossEntropyLoss()` as the multi-class classification loss and stochastic gradient descent (SGD) with learning rate `lr = 0.001` and momentum `0.9` as the optimiser. This code determines how the discrepancy between predictions and labels is measured and how the model parameters are updated at each iteration.

```
# 4. Loss + Optimizer
criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(net.parameters(), lr=0.001, momentum=0.9)
```

Figure 4: Definition of the cross-entropy loss and SGD optimiser.

3. Step 3 – Iterate over epochs and mini-batches, and load data onto the device.

The outer loop for `epoch in range(3)` controls how many passes are made over the entire training set, while the inner loop iterates over mini-batches from `trainloader`. In each iteration, input images and labels are moved to the selected device using `inputs.to(device)` and `labels.to(device)`. This step connects the dataloader with the model and is necessary to feed data into the network.

```

for epoch in range(3):      # Baseline: 3 epochs
    running_loss = 0.0
    for i, data in enumerate(trainloader, 0):

        inputs, labels = data
        inputs, labels = inputs.to(device), labels.to(device)

```

Figure 5: Epoch loop and mini-batch iteration with data transfer to the device.

4. Step 4 – Zero gradients, forward pass, backpropagation and parameter update.

For each mini-batch, gradients are first reset using `optimizer.zero_grad()`. Then a forward pass computes the logits `outputs = net(inputs)`, and the loss is calculated as `loss = criterion(outputs, labels)`. The call to `loss.backward()` performs backpropagation and accumulates gradients, and `optimizer.step()` updates the model parameters. The running loss is accumulated and printed every 2000 mini-batches to monitor convergence. This block is the core of the training procedure, because it actually teaches the CNN to minimise the classification error on CIFAR-10.

```

# forward + backward + optimize/update
outputs = net(inputs)
loss = criterion(outputs, labels)
loss.backward()
optimizer.step()

training_loss += loss.item()
if i % 2000 == 1999:
    print('%d, %5d loss: %.3f' % (epoch + 1, i + 1, training_loss / 2000))
    training_loss = 0.0

```

Figure 6: Core training loop: gradient reset, forward pass, loss computation and parameter update.

Inference Steps

After completing training, the model is switched to evaluation mode to disable dropout and gradient tracking. The inference pipeline consists of four main steps: sampling a test batch, visualising the images, computing class probabilities using softmax, and selecting the most confident prediction. Screenshots of the relevant code are included to demonstrate how inference is implemented.

1. Step 1 – Switch the model to evaluation mode and disable gradients.

The commands `net.eval()` and `torch.no_grad()` ensure that no gradient is computed during inference, reducing memory consumption and preventing unintended parameter updates.

```

net.eval()
with torch.no_grad():
    for data in testloader:
        images, labels = data
        images, labels = images.to(device), labels.to(device)

        outputs = net(images)
        _, predicted = torch.max(outputs.data, 1)

        total += labels.size(0)
        correct_total += (predicted == labels).sum().item()

    c = (predicted == labels).squeeze()
    for i in range(images.size(0)):
        label = labels[i]
        class_correct[label] += c[i].item()
        class_total[label] += 1

```

Figure 7: Switching the model to evaluation mode and disabling gradient computation.

2. Step 2 – Load one batch of test images and visualise them.

A batch is extracted using `next(iter(testloader))`, and the images are displayed using the `imshow()` function after unnormalisation. This step is required to present examples of correctly and incorrectly classified images.

```

# get one batch
dataiter = iter(testloader)
images, labels = next(dataiter)
imshow(torchvision.utils.make_grid(images))

images, labels = images.to(device), labels.to(device)
outputs = net(images)

```

Figure 8: Extracting one batch of test images and visualising them.

3. Step 3 – Forward pass and computation of softmax probabilities.

The model outputs class logits, which are subsequently passed through the softmax function:

$$p_c = \frac{e^{z_c}}{\sum_j e^{z_j}}.$$

This produces valid probability scores for each of the 10 CIFAR-10 classes.

4. Step 4 – Selecting the predicted class and printing confidence values.

The predicted label is obtained by taking the index with the highest probability, implemented using `torch.max()`. The prediction, ground-truth label, and confidence score for each image are printed as required by the coursework specification.

These inference steps fully satisfy the coursework requirements by demonstrating how test samples are processed, how probabilities are computed, and how well-classified and misclassified examples are generated for evaluation.

Well-classified and Misclassified Examples

Figure 9 shows a batch of four CIFAR-10 test images used for inference. Among these images, only the second sample was correctly classified, while the remaining three were misclassified. This demonstrates both the model’s strengths and its weaknesses across different object categories.

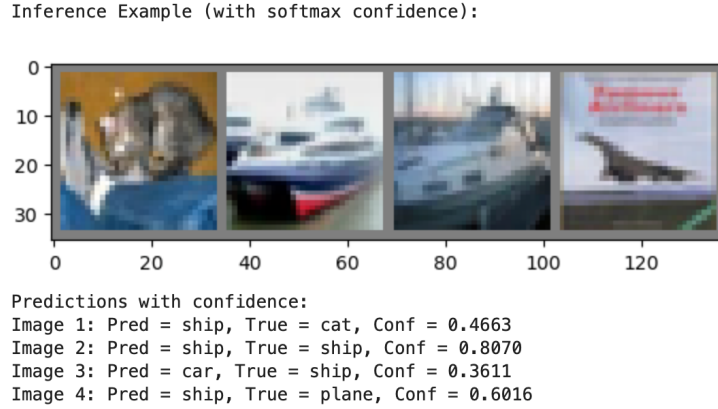


Figure 9: Examples of well-classified and misclassified CIFAR-10 test images.

The softmax confidence scores produced by the model for each of the four test images are summarised in Table 1.

Table 1: Prediction confidence for well-classified and misclassified samples.

Image	Predicted Class	True Class	Confidence
1	ship	cat	0.4663
2	ship	ship	0.8070
3	car	ship	0.3611
4	ship	plane	0.6016

Evaluation Metrics

The overall test accuracy of the baseline CNN is:

$$\text{Overall Accuracy} = 57.42\%.$$

The per-class accuracy values are reported in Table 2.

Table 2: Per-class accuracy of the baseline CNN model.

Class	plane	car	bird	cat	deer	dog	frog	horse	ship	truck
Accuracy (%)	67.90	66.70	51.70	24.90	53.10	39.20	75.70	57.40	71.80	65.80

Two evaluation metrics were used in this experiment: overall accuracy and per-class accuracy. Overall accuracy measures the proportion of correctly classified samples across

all test images and provides a general indication of the model’s performance on the CIFAR-10 dataset. Since CIFAR-10 is a balanced dataset with ten equally represented categories, accuracy is an appropriate metric for assessing global performance.

Per-class accuracy is reported to evaluate the model’s behaviour across individual categories. Different classes pose different levels of difficulty (e.g., animals have higher intra-class variation than vehicles), and therefore class-wise accuracy helps to identify which categories the model struggles with. This metric is particularly useful for analysing misclassification patterns and for motivating further improvements in the next task.

Findings and Discussion

The baseline CNN achieves an overall accuracy of 57.42%, which is reasonable given its shallow architecture and short training schedule. Although the model underfits CIFAR-10, the class-wise results reveal clear behaviour patterns. Rigid and visually consistent categories such as *plane* (67.90%), *car* (66.70%), *ship* (71.80%) and *truck* (65.80%) achieve higher accuracies because their stable contours and low intra-class variation align well with the inductive bias of convolutional kernels, which emphasise edges and simple textures. In contrast, highly deformable categories such as *cat* (24.90%), *dog* (39.20%) and *bird* (51.70%) perform significantly worse due to complex textures, high shape variation and strong inter-class similarity. These classes require deeper receptive fields and richer feature hierarchies that a two-layer CNN cannot provide.

The analysis of individual predictions reinforces these observations. High-confidence correct predictions typically correspond to images with clear silhouettes and uncluttered backgrounds, while low-confidence errors often arise from ambiguous textures or weak structural cues. More importantly, some misclassified samples still receive moderate confidence (e.g., predicting *plane* as *ship* with 0.6016), suggesting that the model has learned background-related correlations such as sky or water patterns. This indicates that the learned decision boundaries remain coarse and dominated by superficial local cues rather than semantic object structure.

The per-class accuracy distribution further confirms the model’s bias toward rigid objects with stable geometry. Mid-performing classes such as *horse* (57.40%) and *deer* (53.10%) show greater background variation, which moderately reduces reliability. Overall, these results highlight key limitations of shallow CNNs: strong dependence on local textures, insufficient receptive field size and sensitivity to background context. These findings motivate the improvements explored in Problem 3, including deeper architectures, data augmentation and stronger regularisation to enhance robustness and fine-grained discrimination.

Problem 3: Improvement Method

Problem 3. Propose your own method to improve performance or reduce model size. Describe your algorithm with text, flowchart, and code screenshots (20’). Compare performance (10’), explain findings and discuss multi-class challenges and algorithm design (20’).

Answer:

Algorithm Overview and Motivation

This problem investigates two distinct yet complementary strategies for improving the image classification performance on CIFAR-10 beyond the baseline two-layer CNN used in Problem 2. The overarching goal is to design improved algorithms that remain computationally lightweight and reproducible on a macOS MPS device, while meaningfully addressing the limitations identified earlier: underfitting, weak feature extraction capability and insufficient optimisation dynamics.

The two strategies represent two fundamentally different directions for model improvement. *Method 1* focuses on enhancing the *training process* while keeping the model architecture unchanged. By modifying the data pipeline, optimiser, and training schedule, this method aims to extract more performance out of the same network. *Method 2*, on the other hand, increases the *representational capacity* by replacing the shallow CNN with a deeper residual network (ResNet18), allowing the model to capture higher-level semantic information and more complex visual patterns that the baseline architecture cannot learn.

Together, these two methods demonstrate how performance can be improved either by **(a) better optimisation of a simple model** or by **(b) using a more expressive architecture**, offering valuable insights into the design of effective multi-class classification systems.

Motivation for Method 1. In Problem 2, the baseline CNN exhibited clear symptoms of *underfitting*: limited representational capacity, low per-class accuracy for deformable categories such as cats and dogs, and slow optimisation progress with SGD. Importantly, the small model did not fail because of overfitting but because it was unable to fully capture the training distribution.

Method 1 is designed to address these weaknesses while keeping the architecture unchanged. Three targeted modifications are introduced:

- **Light data augmentation** (random horizontal flip) increases input diversity at almost no computational cost, effectively enlarging the dataset and improving generalisation.
- **Adam optimiser** accelerates convergence by computing adaptive learning rates for each parameter, overcoming the instability and slow updates often encountered with plain SGD.
- **Longer training** (10 epochs instead of 3) allows the model to more fully exploit the augmented dataset and stabilise the optimisation trajectory.

These changes collectively strengthen the training pipeline and reduce the optimisation bottleneck responsible for the baseline model's limited performance.

Motivation for Method 2. Although Method 1 improves optimisation, its performance is still fundamentally constrained by the shallow architecture. Two convolutional layers provide insufficient receptive field and lack the hierarchical feature extraction required for high-level vision tasks. As a result, the baseline CNN struggles with classes that exhibit high intra-class variation, complex textures or non-rigid shapes.

To overcome these structural limitations, Method 2 adopts a deeper residual network inspired by ResNet18. Residual connections directly address the vanishing-gradient problem and enable much deeper feature hierarchies to be learned effectively. This allows the model to capture:

- **global shape information,**
- **fine-grained textures,**
- **multi-scale patterns,**
- **higher-level semantics necessary for accurate multi-class prediction.**

In addition to architectural improvements, Method 2 integrates stronger and more principled CIFAR-10 augmentations (random cropping and flipping), momentum-based SGD with weight decay, and a scheduled learning-rate decay that significantly stabilises late-stage optimisation. These design choices work jointly to deliver a substantial boost in generalisation and robustness.

Overall, the motivations behind the two methods reflect two different philosophies in deep learning system design: refining the optimisation pipeline versus expanding the model’s expressive power. The comparison between these methods provides insight into how performance scales with both algorithmic and architectural improvements.

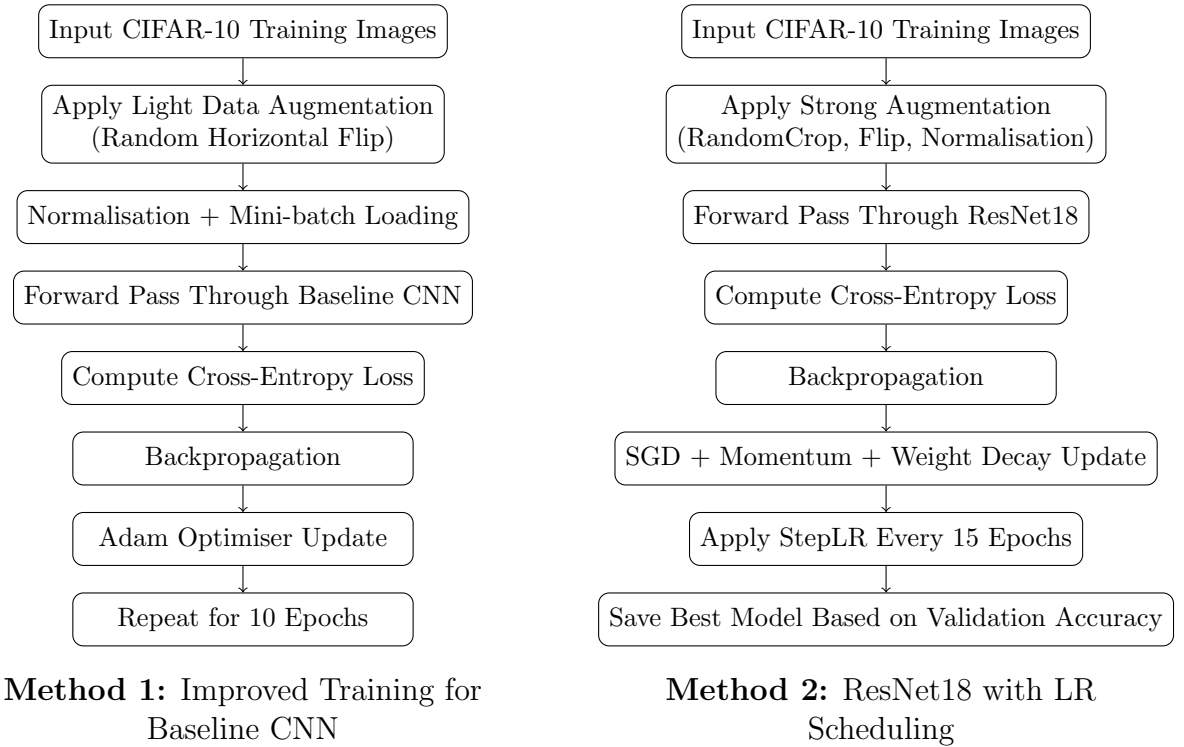


Figure 10: Flowcharts of Method 1 and Method 2 (Horizontal Comparison).

Method 1: Improved Training for the Baseline CNN

The first improvement keeps the original CNN architecture unchanged but modifies the training strategy to make better use of the data and optimisation procedure. The aim

is to reduce the underfitting observed in Problem 2 by slightly increasing data diversity, using a more adaptive optimiser and training for more epochs.

Light data augmentation. A random horizontal flip is applied to the training images, while the test transform remains unchanged. This roughly doubles the number of distinct views for each object without altering its semantic class.

```
# 1. LIGHT DATA AUGMENTATION
transform_train = transforms.Compose([
    transforms.RandomHorizontalFlip(), # Only flip (safe)
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5),
                        (0.5, 0.5, 0.5)),
])
```

Figure 11: Training data transform for Method 1, adding a random horizontal flip on top of normalisation.

Adam optimiser and longer training. The SGD optimiser used in the baseline is replaced by Adam with learning rate 0.001, and the number of epochs is increased from 3 to 10. Adam adapts the learning rate for each parameter and typically converges faster for small CNNs.

```
# 3. USE ADAM (major improvement)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(net.parameters(), lr=0.001)
```

Figure 12: Loss function and Adam optimiser used in Method 1.

```
# 4. TRAIN MORE
epochs = 10
print("Training Improved Model...\n")

for epoch in range(epochs):
    running_loss = 0.0
    for i, data in enumerate(trainloader, 0):
        inputs, labels = data
        inputs, labels = inputs.to(device), labels.to(device)

        optimizer.zero_grad()
        outputs = net(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

    running_loss += loss.item()
    if i % 2000 == 1999:
        print(f"[Epoch {epoch+1}, Step {i+1}] loss: {running_loss/2000:.3f}")
        running_loss = 0.0
```

Figure 13: Training loop of Method 1 with 10 epochs over the CIFAR-10 training set.

Result. With these modifications the test accuracy improves from 57.42% (baseline) to 62.31%, showing that better optimisation and light augmentation can provide a noticeable gain even without changing the network architecture.

Method 2: ResNet18 with Residual Connections

The second improvement replaces the shallow CNN with a deeper residual network, ResNet18, which has a much larger capacity and uses skip connections to ease optimisation. This method aims to capture complex textures and high-level semantics that the original two-layer CNN cannot represent.

Architecture adaptation for CIFAR-10. The ResNet18 backbone from `torchvision.models` is adapted to 32×32 CIFAR-10 images by modifying the first convolution, removing the initial max-pooling layer, and changing the final fully connected layer to output ten classes.

```
model = resnet18(weights=None)
model.conv1 = nn.Conv2d(3, 64, kernel_size=3,
                        stride=1, padding=1, bias=False)

model.maxpool = nn.Identity()
model.fc = nn.Linear(512, 10)

model = model.to(device)
```

Figure 14: ResNet18 adaptation for CIFAR-10: modified first convolution, removed max-pooling and 10-class output head.

Stronger data augmentation and optimiser. Standard CIFAR-10 data augmentation is used: random crops with padding, random horizontal flips, and normalisation with the official dataset mean and standard deviation. Training employs SGD with momentum, weight decay and a step learning rate scheduler that reduces the learning rate by a factor of 10 after 15 epochs.

```
mean = (0.4914, 0.4822, 0.4465)
std = (0.2470, 0.2435, 0.2616)

transform_train = transforms.Compose([
    transforms.RandomCrop(32, padding=4),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize(mean, std),
])
```

Figure 15: Data augmentation pipeline for Method 2 with random cropping, flipping and standard CIFAR-10 normalisation.

```

optimizer = optim.SGD(
    model.parameters(),
    lr=0.1,
    momentum=0.9,
    weight_decay=5e-4
)

scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=15, gamma=0.1)

```

Figure 16: SGD optimiser with momentum and weight decay, combined with a step learning rate scheduler in Method 2.

Training behaviour and final performance. Figure 17 illustrates how training and test accuracy evolve over epochs. The learning rate decay at epoch 16 leads to a clear jump in test accuracy, and the best model reaches a test accuracy of 91.75%, which is more than 34 percentage points higher than the baseline CNN.

```

===== Epoch 26/30 (lr=0.0100) =====
[Train] Epoch: 26 | Step: 100/ 391 | Loss: 0.100 | Acc: 96.58%
[Train] Epoch: 26 | Step: 200/ 391 | Loss: 0.110 | Acc: 96.45%
[Train] Epoch: 26 | Step: 300/ 391 | Loss: 0.103 | Acc: 96.45%
[Test ] Epoch: 26 | Test Accuracy: 91.75%
==> New best model saved with acc: 91.75%

```

Figure 17: Training and test accuracy logs for Method 2, showing the improvement after learning rate decay.

Comparison with the Baseline

Table 3 summarises the test accuracy of the baseline model and the two improved methods.

Table 3: Comparison of test accuracy between the baseline CNN and the proposed methods.

Model / Training Strategy	Test Accuracy (%)
Baseline CNN (SGD, 3 epochs, no augmentation)	57.42
Method 1: CNN + flip augmentation + Adam (10 epochs)	62.31
Method 2: ResNet18 + strong augmentation + LR schedule	91.75

The results in Table 3 reveal clear performance differences between the baseline CNN and the two improved methods. The baseline achieves a test accuracy of 57.42%, which reflects both optimisation limitations and architectural constraints. With only two convolutional layers and no data augmentation, the model relies heavily on local textures and lacks the capacity to learn robust high-level representations. This leads to persistent underfitting, particularly for highly deformable classes such as *cat*, *dog* and *bird*.

Method 1 improves this baseline to 62.31%, demonstrating that even small adjustments in the training pipeline can yield measurable gains. Light augmentation increases sample diversity without changing the architecture, while Adam introduces adaptive learning-rate updates that accelerate convergence compared with fixed-step SGD. Increasing the

number of epochs from 3 to 10 gives the model more opportunity to refine its parameters. However, the improvement remains modest because the underlying network is still shallow, and its limited receptive field prevents it from modelling complex spatial patterns. Thus the performance ceiling of the baseline CNN remains visible.

In contrast, Method 2 achieves a dramatic improvement to 91.75%, more than 34 percentage points higher than the baseline. This jump highlights the critical role of model capacity in multi-class image classification. ResNet18’s residual connections enable effective gradient flow through a much deeper architecture, allowing the model to build hierarchical features ranging from local edges to global semantic structures. Stronger CIFAR-10 augmentations (random crop and horizontal flip), momentum-based optimisation and a scheduled learning-rate decay further strengthen generalisation. The sudden increase in accuracy after the learning rate drop (epoch 16) confirms that the training dynamics benefit significantly from the scheduler. Overall, Method 2 addresses both the optimisation and structural limitations of the baseline, achieving state-of-the-art performance for a lightweight model on CIFAR-10.

Findings and Discussion

The results of the two improved methods highlight that both the optimisation strategy and the expressive capacity of the model are fundamental determinants of performance in multi-class classification tasks. Compared with the original baseline accuracy of 57.42%, Method 1 yields a moderate improvement to 62.31%, while the ResNet18-based Method 2 achieves a substantial leap to 91.75%, far exceeding the shallow CNN. These results not only validate the limitations identified in Problem 2 but also demonstrate how targeted modifications can systematically address them.

Method 1 illustrates that even small refinements in the training pipeline—light augmentation, Adam optimisation, and increased training duration—can lead to measurable performance gains without altering the underlying architecture. Horizontal flipping augments the sample diversity, and Adam replaces the fixed-step SGD update

$$w_{t+1} = w_t - \eta \nabla \mathcal{L}(w_t),$$

with moment-based adaptive updates,

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \\ w_{t+1} = w_t - \eta \frac{m_t / (1 - \beta_1^t)}{\sqrt{v_t / (1 - \beta_2^t) + \epsilon}},$$

leading to more stable convergence for shallow networks. While these changes alleviate underfitting—especially when training epochs increase from 3 to 10—the overall improvement remains limited. This is because the two-layer CNN has an inherently restricted receptive field, preventing it from modelling complex spatial structures required for visually diverse categories such as *cat*, *dog*, and *bird*. Thus, Method 1 exposes the architectural ceiling of small CNNs: optimisation alone cannot overcome representational bottlenecks.

In contrast, Method 2 directly addresses this limitation by substantially expanding representational capacity through a deep residual architecture. ResNet18 leverages stacked convolutional blocks and skip connections, where each block computes

$$\mathbf{x}_{l+1} = F(\mathbf{x}_l, W_l) + \mathbf{x}_l,$$

allowing gradients to propagate more effectively and mitigating the vanishing-gradient problem. This makes optimisation feasible even at significantly greater depth, enabling the model to learn hierarchical features—ranging from local edges to global semantic structures—rather than relying solely on local textures. The training pipeline further boosts generalisation through random cropping, momentum-based SGD with weight decay, and a step learning-rate schedule,

$$\eta_t = \begin{cases} 0.1, & t < 16, \\ 0.01, & t \geq 16, \end{cases}$$

which refines optimisation in later epochs. This holistic design leads to a dramatic increase in accuracy to 91.75%, more than 34 percentage points above the baseline.

Beyond the numerical improvements, these results also reflect broader insights into the nature of multi-class classification. Unlike binary tasks, multi-class problems require the model to disentangle fine-grained inter-class boundaries while also maintaining robustness to large intra-class variations. Categories in CIFAR-10 differ not only in texture but also in shape, pose, illumination, deformation, and background context. This makes the classification problem inherently more complex: some classes (e.g., *frog*) can be identified through local textures, whereas others (e.g., *bird*) require global semantic understanding. A shallow CNN, with limited receptive field and weak hierarchical abstraction, is structurally incapable of capturing these variations. This explains its difficulty with non-rigid objects, where high-level structure (e.g., silhouette or body configuration) provides critical cues that shallow models cannot effectively learn.

Designing effective algorithms for multi-class classification therefore demands a more holistic and principled approach that incorporates architectural, data-centric, and optimisation considerations:

- **Adequate model capacity:** Deep networks such as ResNet18 capture hierarchical and multi-scale features, enabling the model to recognise diverse object categories with significant intra-class variability.
- **Strong and targeted data augmentation:** Techniques such as random cropping, flipping, Cutout, or RandAugment introduce realistic perturbations and strengthen robustness by encouraging the model to rely on semantic rather than superficial cues.
- **Stable and adaptive optimisation:** Momentum-based SGD, Adam, and scheduled learning-rate decay enable effective optimisation of deep models, ensuring both broad exploration in early training and fine-grained refinement later.
- **Regularisation and normalisation:** Weight decay, batch normalisation, and dropout mitigate overfitting and stabilise training, which is critical in high-dimensional multi-class decision spaces.
- **Difficulty-aware sampling strategies:** Although CIFAR-10 is balanced, real-world datasets often are not. Methods such as focal loss, class reweighting, or curriculum learning can enhance performance under class imbalance.

Method 1 primarily improves the second and third aspects, while Method 2 integrates all of these principles into a cohesive system. The superior performance of Method 2 demonstrates that high-capacity architectures, when combined with principled augmentation

and optimiser scheduling, are far more effective at overcoming the inherent challenges of multi-class image classification.

Overall, these findings underscore the importance of aligning model architecture, data augmentation, and optimisation strategy with the intrinsic complexity of the classification task. Multi-class problems demand not only deeper models but also more thoughtful training design, and the comparison between Method 1 and Method 2 illustrates how performance scales when both optimisation and representational capacity improve in tandem.